Architectures logicielles et matérielles

Cours 8 : Circuits séquentiels, machines à états

Laurence Pierre

Circuits séquentiels

- Rappel : un circuit combinatoire
  - représente une **fonction booléenne**
  - en négligeant le temps de propagation, fournit un résultat au temps t en fonction d'entrées stables au temps t
  - n'a *pas de faculté de mémorisation de son "état"* pour **adapter** son comportement selon l'état courant (et entrées)
- … mais **mémoriser son état** est souvent indispensable
  ⇒ circuits **séquentiels**

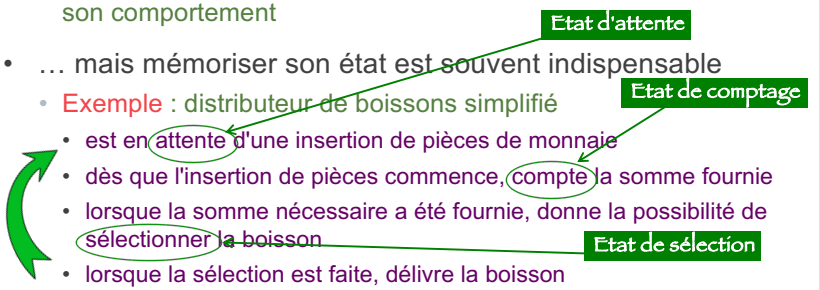
Circuits séquentiels

- Rappel : un circuit combinatoire
  - représente une **fonction booléenne**
  - en négligeant le temps de propagation, fournit un résultat au temps t en fonction d'entrées stables au temps t
  - n'a *pas de faculté de mémorisation de son "état"* pour **adapter** son comportement selon l'état courant (et entrées)
- … mais mémoriser son état est souvent indispensable
  - Exemple : distributeur de boissons simplifié
    - est en attente d'une insertion de pièces de monnaie
    - dès que l'insertion de pièces commence, compte la somme fournie
    - lorsque la somme nécessaire a été fournie, donne la possibilité de sélectionner la boisson
    - lorsque la sélection est faite, délivre la boisson



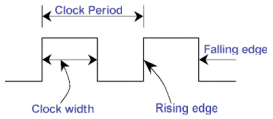
Circuits séquentiels

- Rappel : un circuit combinatoire
  - représente une **fonction booléenne**
  - en négligeant le temps de propagation, fournit un résultat au temps t en fonction d'entrées stables au temps t
  - n'a *pas de faculté de mémorisation de son "état"* pour **adapter** son comportement
- … mais mémoriser son état est souvent indispensable
  - Exemple : distributeur de boissons simplifié
    - est en attente d'une insertion de pièces de monnaie
    - dès que l'insertion de pièces commence, compte la somme fournie
    - lorsque la somme nécessaire a été fournie, donne la possibilité de sélectionner la boisson
    - lorsque la sélection est faite, délivre la boisson



Circuits séquentiels

- Pour réaliser un tel comportement, nous avons recours à des circuits
  - séquentiels : leur comportement et leur résultat à un temps t dépendent des stimuli exercés à ce temps t, mais aussi de ***l'état*** *courant* du système (lui-même obtenu selon la suite de stimuli exercés, à partir d'un état initial)
  - généralement synchrones : l'évolution du système est cadencée par un signal périodique, le *signal* ***d'horloge***, qui induit une discrétisation du temps



Comment conçoit-on et synthétise-t-on ces circuits ?

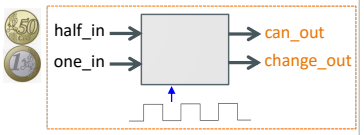
Conception des circuits séquentiels

- Leur conception nécessite de
  - **modéliser** le comportement souhaité : le modèle couramment utilisé est celui d'un automate (ou **FSM**, Finite State Machine, machine à états finis)
    - Définit l'évolution du système selon des *états* (symboliques) possibles, et des *transitions* entre ces états
  - **transformer** cette spécification (synthétiser) en un circuit séquentiel synchrone
    - Dans ce circuit, des composants spécifiques permettront de mémoriser l'état du système

Nous allons voir
1. la **modélisation** sous forme de machines de Moore et de Mealy
2. la méthode de **synthèse** à partir de ce modèle

① Modélisation par automate

- Exemple : distributeur de boissons rudimentaire, 1 seule boisson possible (à 1.5 €), n'accepte que des pièces de 1€ et 0.5€

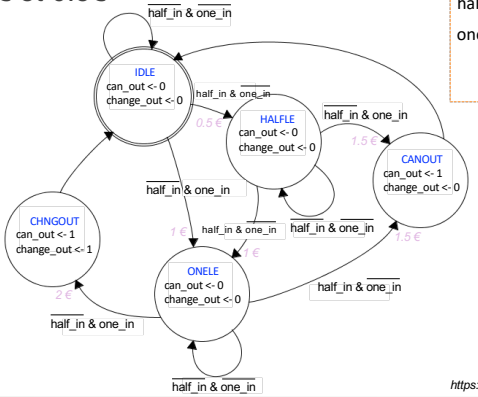


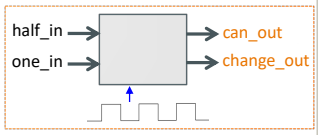


https://github.com/mahmoud-gawad/vending-machine-fsm

① Modélisation par automate

- Exemple : distributeur de boissons rudimentaire, 1 seule boisson possible (à 1.5 €), n'accepte que des pièces de 1€ et 0.5€



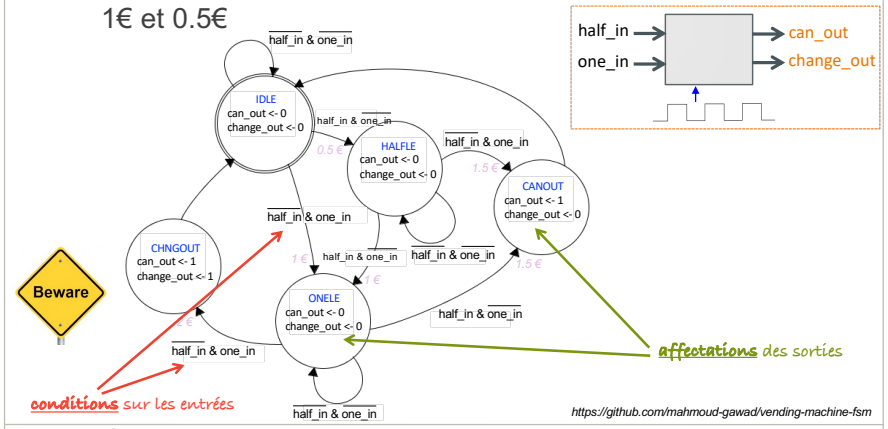




https://github.com/mahmoud-gawad/vending-machine-fsm

① Modélisation par automate

- Exemple : distributeur de boissons rudimentaire, 1 seule boisson possible (à 1.5 €), n'accepte que des pièces de 1€ et 0.5€



https://github.com/mahmoud-gawad/vending-machine-fsm

① Modélisation par automate

- Exemple : distributeur de boissons rudimentaire, 1 seule boisson possible (à 1.5 €), n'accepte que des pièces de 1€ et 0.5€

Votre modélisation devra toujours représenter une machine **déterministe et complète** :

Pour chaque état et combinaison faisable des entrées, l'état suivant doit être défini et unique

https://github.com/mahmoud-gawad/vending-machine-fsm

① Modélisation par automate

- Exemple : distributeur de boissons rudimentaire, 1 seule boisson possible (à 1.5 €), n'accepte que des pièces de 1€ et 0.5€

5 états symboliques : ?

2 entrées booléennes ⇒ ensemble des valeurs possibles = ?

2 sorties booléennes ⇒ ensemble des valeurs possibles = ?

Machine de Moore

- L'automate de l'exemple précédent est une *Machine de Moore* : la valeur courante des sorties dépend uniquement de l'état courant
- Machine de Moore : (Q, I, O, t, f, q_0)
  - Q : ensemble d'états (symboliques)
  - I : ensemble des valeurs possibles des entrées
  - O : ensemble des valeurs possibles des sorties
  - t : fonction de transition $t : Q \times I \rightarrow Q$
  - f : fonction de sortie $f : Q \rightarrow O$
  - q_0 : état initial

Exemple ⇓

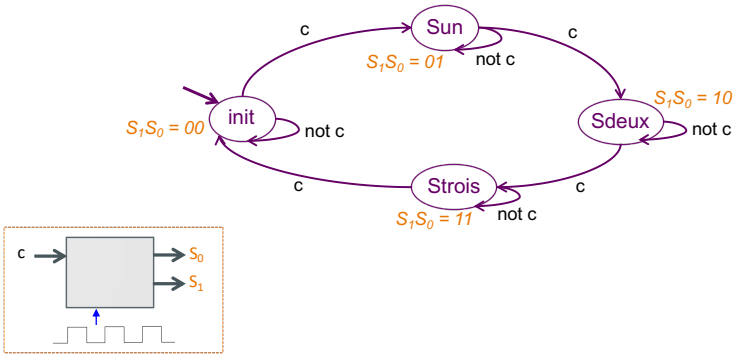
Q = {Idle, Halfle, Onele, Canout, Chngout}

$I = \{ 0, 1 \}^2$

$O = \{ 0, 1 \}^2$

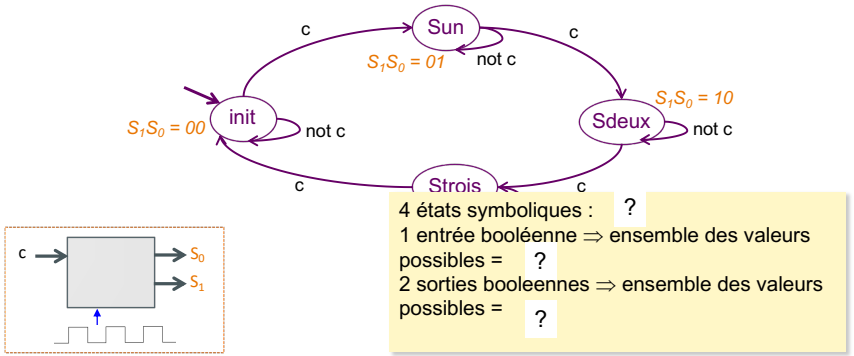
Machine de Moore

- Exemple 2 : compteur 2 bits avec une entrée de contrôle c
  - s'incrémente si l'entrée c vaut 1
  - et garde la valeur courante si (tant que) l'entrée c vaut 0



Machine de Moore

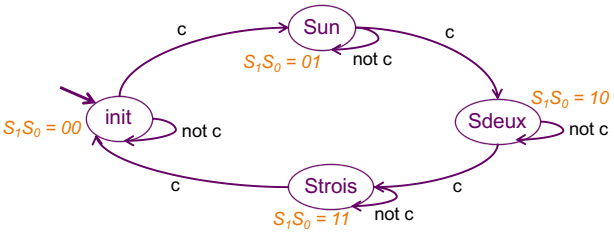
- Exemple 2 : compteur 2 bits avec une entrée de contrôle c
  - s'incrémente si l'entrée c vaut 1
  - et garde la valeur courante si (tant que) l'entrée c vaut 0



4 états symboliques : ?
1 entrée booléenne ⇒ ensemble des valeurs possibles = ?
2 sorties booleennes ⇒ ensemble des valeurs possibles = ?

Machine de Moore

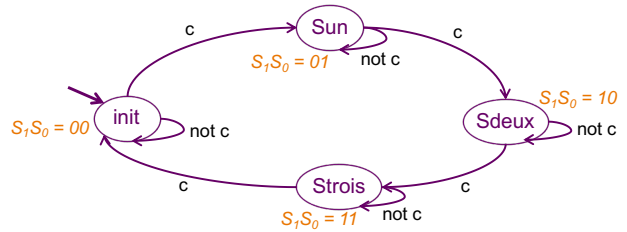
- Exemple 2 : compteur 2 bits avec une entrée de contrôle c
  - s'incrémente si l'entrée c vaut 1
  - et garde la valeur courante si (tant que) l'entrée c vaut 0



- Machine de Moore :
  - Q = { init, Sun, Sdeux, Strois } q_0 = init
  - I = { 0, 1 } O = { 0, 1 }^{2}

Machine de Moore

- Exemple 2 : compteur 2 bits avec une entrée de contrôle c
  - s'incrémente si l'entrée c vaut 1
  - et garde la valeur courante si (tant que) l'entrée c vaut 0

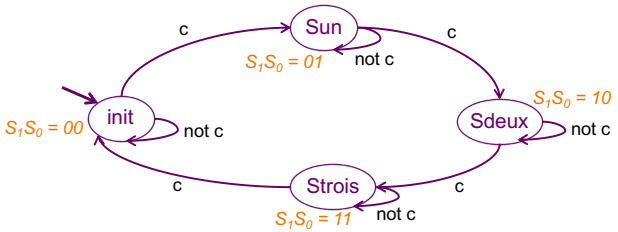


- Machine de Moore :
  - **t** : Q x I → Q

t(init, 0) = ? t(init, 1) = ?
t(Sun, 0) = t(Sun, 1) =
t(Sdeux, 0) = t(Sdeux, 1) =
t(Strois, 0) = t(Strois, 1) =

Machine de Moore

- Exemple 2 : compteur 2 bits avec une entrée de contrôle c
  - s'incrémente si l'entrée c vaut 1
  - et garde la valeur courante si (tant que) l'entrée c vaut 0



- Machine de Moore :
  - **f** : Q → O

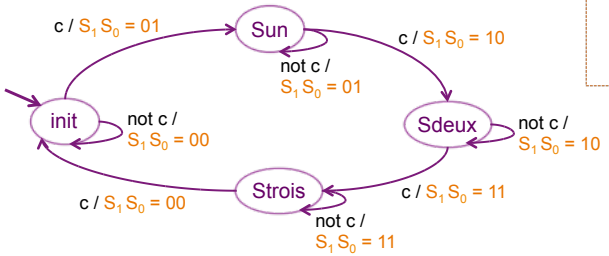
f(init) =
f(Sun) =
f(Sdeux) =
f(Strois) =
?

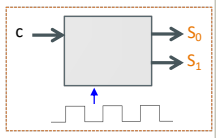
Machine de Mealy

- La spécification du comportement d'un circuit séquentiel peut aussi se faire sous la forme d'une *Machine de Mealy* : la valeur courante de ses sorties dépend de l'état courant ET de la valeur courante de ses entrées
- Machine de Mealy : (Q, I, O, t, f, q_0)
  - Q : ensemble d'états (symboliques)
  - I : ensemble des valeurs possibles des entrées
  - O : ensemble des valeurs possibles des sorties
  - t : fonction de transition t : Q x I → Q
  - f : fonction de sortie f : Q x I → O
  - q_0 : état initial

Machine de Mealy

- Exemple : une version Mealy du compteur 2 bits avec une entrée de contrôle c pourrait être la suivante

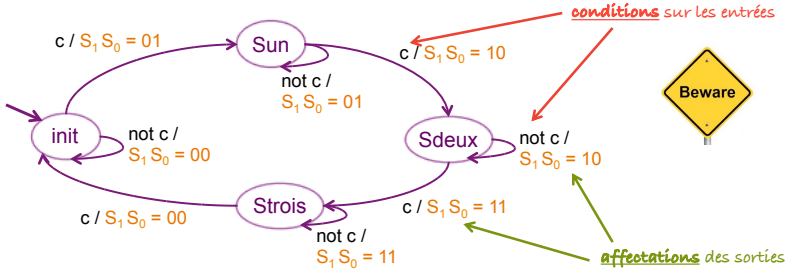




- dans cette version, les sorties sont affectées dès réception de l'entrée de contrôle (et pas dans l'état atteint)

Machine de Mealy

- Exemple : une version Mealy du compteur 2 bits avec une entrée de contrôle c pourrait être la suivante



- dans cette version, les sorties sont affectées dès réception de l'entrée de contrôle (et pas dans l'état atteint)

② Synthèse en circuit séquentiel

- Une telle spécification sous forme d'automate pourra être transformée en un circuit séquentiel synchrone
  - Les fonctions de **transition** (t) et de **sortie** (f) vont être traduites en fonctions booléennes (blocs combinatoires) permettant de **calculer l'état suivant et les valeurs des sorties**
  - L'état sera mis à jour *sur les fronts (généralement) montants de l'horloge*. On prévoira de plus un signal de reset.
- Comment se traduit la mémorisation de l'état du système ?

→ Par des composants de mémorisation (bascules), détaillés en annexe

- Et comment produire des fonctions *booléennes* alors que les états sont *symboliques* (ex. init, Sun, Sdeux,…) ?

→ ?

② Synthèse en circuit séquentiel

- Une telle spécification sous forme d'automate pourra être transformée en un circuit séquentiel synchrone
  - Les fonctions de **transition** (t) et de **sortie** (f) vont être traduites en fonctions booléennes (blocs combinatoires) permettant de **calculer l'état suivant et les valeurs des sorties**
  - L'état sera mis à jour *sur les fronts (généralement) montants de l'horloge*. On prévoira de plus un signal de reset.
- Comment se traduit la mémorisation de l'état du système ?

→ Par des composants de mémorisation (bascules), détaillés en annexe

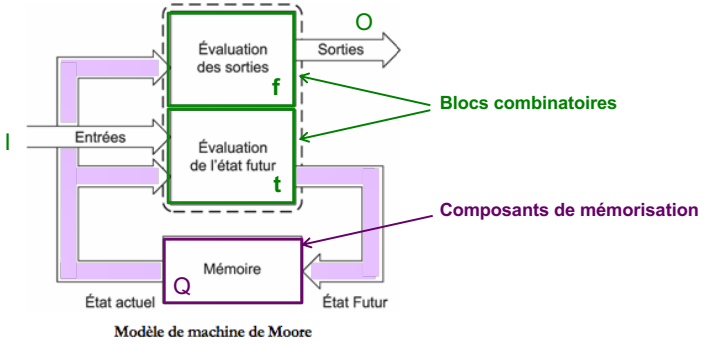
- Et comment produire des fonctions *booléennes* alors que les états sont *symboliques* (ex. init, Sun, Sdeux,…) ?

→ Il faut décider d'un codage (numérique) de l'état

Synthèse en circuit séquentiel

- Machine de Moore : le circuit produit aura la structure suivante

t : *fonction de transition* t : Q x I → Q
f : *fonction de sortie* f : Q → O



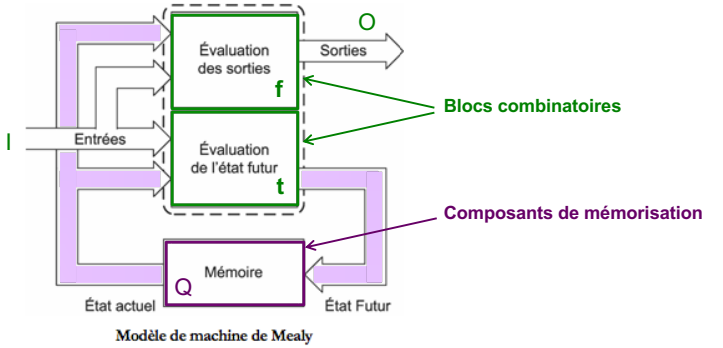
Modèle de machine de Moore

http://www.circuits-logiques.polymtl.ca/help/Chapitre06.pdf

Synthèse en circuit séquentiel

- Machine de Mealy : le circuit produit aura la structure suivante

t : *fonction de transition* t : Q x I → Q
f : *fonction de sortie* f : Q x I → O



Modèle de machine de Mealy

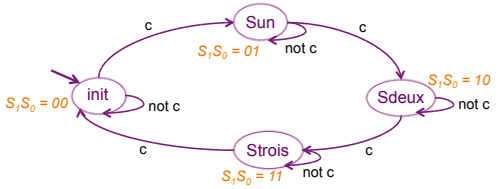
http://www.circuits-logiques.polymtl.ca/help/Chapitre06.pdf

Synthèse en circuit séquentiel

- Il reste à générer **les deux blocs combinatoires** : calcul des sorties (**f**), et calcul de l'état futur (**t**), compte tenu d'un choix de **codage** de l'état symbolique
- Principe :
  - Etape 1 : à partir de l'automate, mise sous forme tabulaire des fonctions f et t (1)
  - Etape 2 : choix d'un codage pour l'état (en particulier, il détermine le nombre de bits de mémorisation qui seront nécessaires) (2)
  - Etape 3 : à partir des tables de vérité dans lesquelles l'état symbolique a été remplacé par son codage, dérivation des expressions algébriques associées à f et à t (voir cours 7) (3)

Synthèse en circuit séquentiel

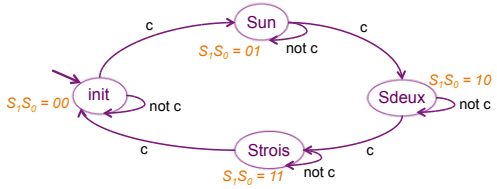
- Exemple 1 : compteur 2 bits



  - + nous prenons la **numérotation** suivante pour l'état symbolique (i.e. **ordre** sur les états symboliques) :
    - init = 0
    - Sun = 1
    - Sdeux = 2
    - Strois = 3

Synthèse en circuit séquentiel

- Exemple 1 : compteur 2 bits



t(init, 0) = init
t(Sun, 0) = Sun
t(Sdeux, 0) = Sdeux
t(Strois, 0) = Strois
t(init, 1) = Sun
t(Sun, 1) = Sdeux
t(Sdeux, 1) = Strois
t(Strois, 1) = init

c	q	q'
0	init	init
0	Sun	Sun
0	Sdeux	Sdeux
0	Strois	Strois
1	init	Sun
1	Sun	Sdeux
1	Sdeux	Strois
1	Strois	init

1 Tables

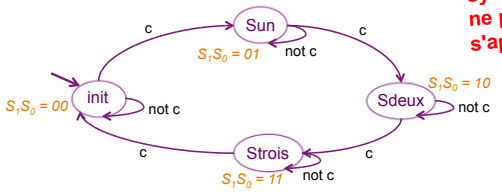
Etat q	S_1 S_0
init	00
Sun	01
Sdeux	10
Strois	11

f(init) = (0,0)
f(Sun) = (0,1)
f(Sdeux) = (1,0)
f(Strois) = (1,1)

Synthèse en circuit séquentiel

- Exemple 1 : compteur 2 bits

Mais pourquoi la méthode de **synthèse** décrite au cours 7, slide 19, **ne peut-elle pas (encore) s'appliquer avec de telles tables ??**



1 Tables

Etat q	S_1 S_0
init	00
Sun	01
Sdeux	10
Strois	11

c	q	q'
0	init	init
0	Sun	Sun
0	Sdeux	Sdeux
0	Strois	Strois
1	init	Sun
1	Sun	Sdeux
1	Sdeux	Strois
1	Strois	init

Synthèse en circuit séquentiel

- Exemple 1 : compteur 2 bits
  - En choisissant un **codage** binaire classique **et** en considérant le choix de **numérotation** du slide 69, on a init (0) = 00, Sun (1) = 01, Sdeux (2) = 10, et Strois (3) = 11 (2 bits pour coder 4 valeurs possibles)

2 Tables avec **codage**

q_1 q_0	S_1 S_0
00	00
01	01
10	10
11	11

c	q_1 q_0	q'_1 q'_0
0	00	00
0	01	01
0	10	10
0	11	11
1	00	01
1	01	10
1	10	11
1	11	00

Synthèse en circuit séquentiel

- Exemple 1 : compteur 2 bits
  - En choisissant un **codage** binaire classique, on a init = 00, Sun = 01, Sdeux = 10, Strois = 11

2 Tables avec codage

q_1 q_0	S_1 S_0
00	00
01	01
10	10
11	11

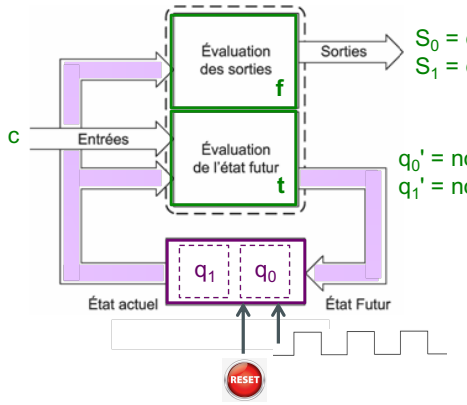
c	q_1 q_0	q'_1 q'_0
0	00	00
0	01	01
0	10	10
0	11	11
1	00	01
1	01	10
1	10	11
1	11	00

3 Equations

- $S_0 = \text{not}(q_1).q_0 + q_1.q_0 = q_0$ $\quad S_1 = q_1.\text{not}(q_0) + q_1.q_0 = q_1$
- $q_0' = \text{not}(c).\text{not}(q_1).q_0 + \text{not}(c).q_1.q_0 + c.\text{not}(q_1).\text{not}(q_0) + c.q_1.\text{not}(q_0)$
  $= \text{not}(c).q_0 + c.\text{not}(q_0)$
- $q_1' = \text{not}(c).q_1.\text{not}(q_0) + \text{not}(c).q_1.q_0 + c.\text{not}(q_1).q_0 + c.q_1.\text{not}(q_0)$
  $= \text{not}(c).q_1 + c.(q_0 \text{ xor } q_1)$

Synthèse en circuit séquentiel

- Exemple 1 : compteur 2 bits

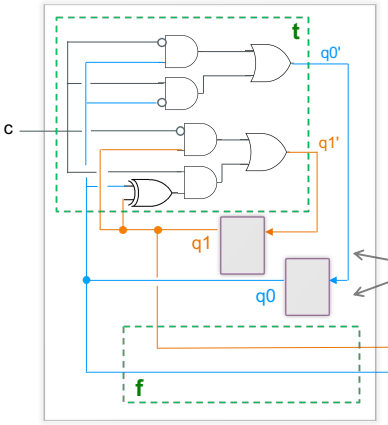


$S_0 = q_0$
$S_1 = q_1$

$q_0' = \text{not}(c).q_0 + c.\text{not}(q_0)$
$q_1' = \text{not}(c).q_1 + c.(q_0 \text{ xor } q_1)$

Synthèse en circuit séquentiel

- Exemple 1 : compteur 2 bits - Netlist détaillée

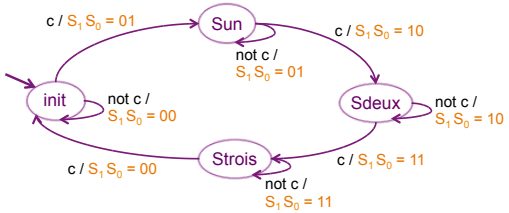


$q_0' = \text{not}(c).q_0 + c.\text{not}(q_0)$
$q_1' = \text{not}(c).q_1 + c.(q_0 \text{ xor } q_1)$

$S_0 = q_0$
$S_1 = q_1$

Synthèse en circuit séquentiel

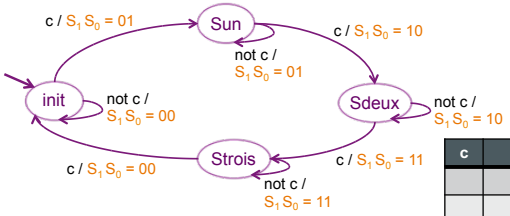
- Exemple 2 : version Mealy de ce compteur 2 bits (sorties affectées dès réception de l'entrée)



  - avec la même **numérotation** pour l'état symbolique :
    - init = 0
    - Sun = 1
    - Sdeux = 2
    - Strois = 3

Synthèse en circuit séquentiel

- Exemple 2 : version Mealy de ce compteur 2 bits (sorties affectées dès réception de l'entrée)



1 Tables

c	q	S_1 S_0

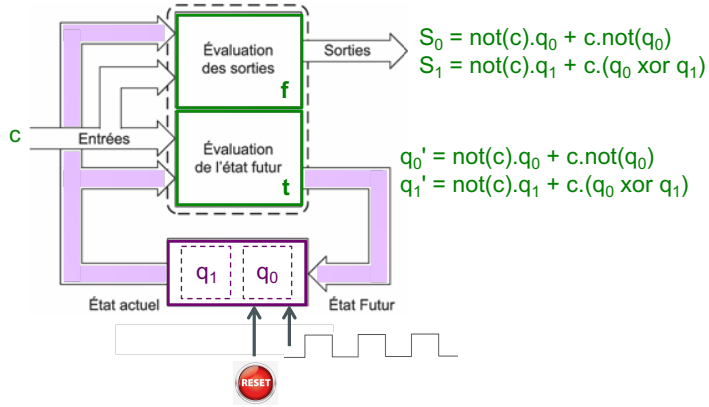
?

c	q	q'

En **exercice** à préparer : comme pour l'exemple précédent, finir l'étape 1 et appliquer les étapes 2 et 3 du slide 68

Synthèse en circuit séquentiel

- Exemple 2 : version Mealy de ce compteur 2 bits



$S_0 = not(c).q_0 + c.not(q_0)$
$S_1 = not(c).q_1 + c.(q_0 \text{ xor } q_1)$

$q_0' = not(c).q_0 + c.not(q_0)$
$q_1' = not(c).q_1 + c.(q_0 \text{ xor } q_1)$

Codage de l'état (étape 2)

- Parmi les codages couramment utilisés
  - Le codage binaire classique (numération de position), pour lequel $\log_2$(nb_états) bits de mémorisation suffisent
  - Le codage de Gray, qui nécessite aussi $\log_2$(nb_états) bits de mémorisation (brevet 1953)
  - Le codage "one hot", qui nécessite nb_états bits de mémorisation

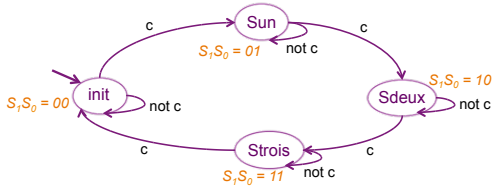

Frank Gray

- Pour une même **numérotation** (en décimal) des états, on a :

Décimal	Binaire	Gray	One hot
0	000	000	00000001
1	001	001	00000010
2	010	011	00000100
3	011	010	00001000
4	100	110	00010000
5	101	111	00100000
6	110	101	01000000
7	111	100	10000000

Codage de l'état

- Exemple : compteur 2 bits (exemple 1)



1 Tables
(idem slide 70)

Etat q	S_1 S_0
init	00
Sun	01
Sdeux	10
Strois	11

c	q	q'
0	init	init
0	Sun	Sun
0	Sdeux	Sdeux
0	Strois	Strois
1	init	Sun
1	Sun	Sdeux
1	Sdeux	Strois
1	Strois	init

Codage de l'état

- Exemple : compteur 2 bits
  - Selon un **codage** de Gray, init = ??, Sun = ??, Sdeux = ??, et Strois = ??

2 Tables avec codage

q_1 q_0	S_1 S_0
?	

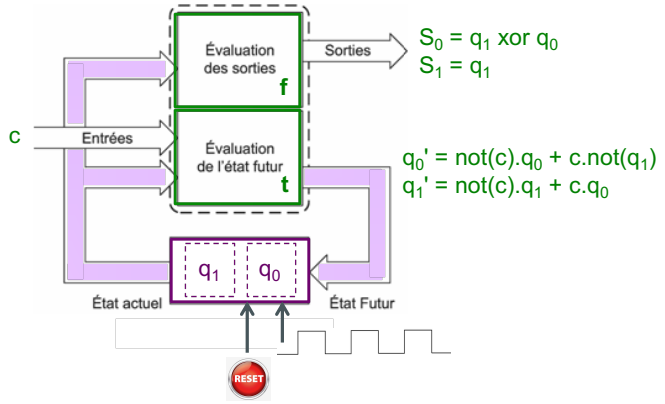
c	q_1 q_0	q'_1 q'_0
0		
0		
0		
0		
1		
1		
1		
1		

?

3 Equations
- S_0 =
- S_1 =
- q_0' =
- q_1' =

Codage de l'état

- Exemple : compteur 2 bits

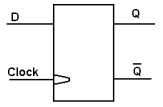


$S_0 = q_1$ xor q_0
$S_1 = q_1$

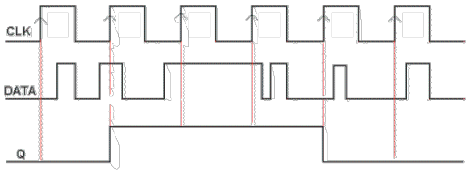
$q_0' = \text{not}(c).q_0 + c.\text{not}(q_1)$
$q_1' = \text{not}(c).q_1 + c.q_0$

Mémorisation de l'état : D flip-flop

- Les composants de mémorisation seront des D flip-flops, chargés sur fronts montants de l'horloge, de symbole :



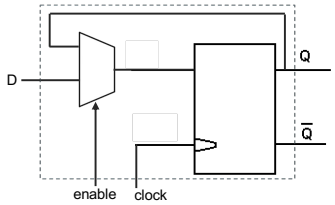
- Exemple du comportement du DFF (voir détails sur sa conception et son comportement en annexe) :



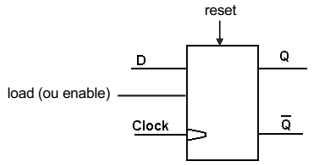
http://www.hobbyprojects.com/flip_flop/edge_triggered_flip-flops.html

Mémorisation de l'état : D flip-flop

- Il en existe diverses variantes :
  - Avec signal de **chargement** (enable, ou load)



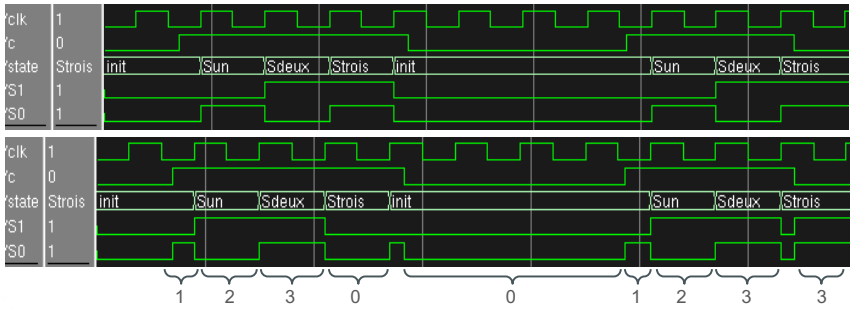
  - Et même avec signal de **reset** (remise à 0)



Compléments

I.Comportement des sorties

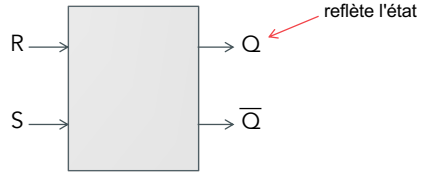
- Attention : les sorties d'une machine de Moore sont disponibles sur (juste après) les fronts d'horloge, mais potentiellement n'importe quand dans le cycle pour une machine de Mealy
  - Exemple : chronogrammes pour nos 2 versions de compteur



II.Mémorisation - Bascules

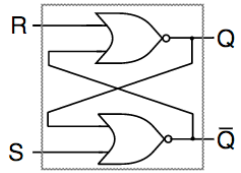
- Comment mémoriser ? => besoin d'un élément matériel qui mémorise (conserve) une valeur
  - Il faut donc qu'il puisse rester dans un état stable associé à cette valeur
  - Une bascule (latch) est en fait un élément *bistable* : elle possède 2 états stables, états logiques 0 ou 1
- Premier type de bascule : **la bascule RS**

R (**reset**) à 1 ⇒
placer dans l'état **0**
S (**set**) à 1 ⇒
placer dans l'état **1**
R et S à 0 ⇒
conserver la valeur placée



Bascules

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)



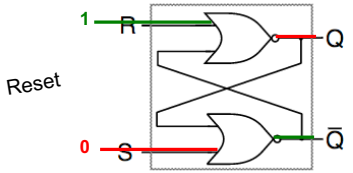
  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

Bascules

a	b	a nor b
0	0	1
0	1	0
1	0	0
1	1	0

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)



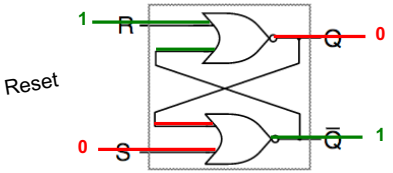
  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

Bascules

a	b	a nor b
0	0	1
0	1	0
1	0	0
1	1	0

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)



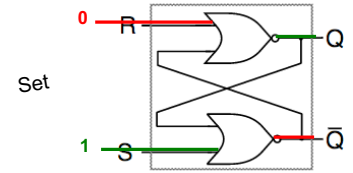
  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

Bascules

a	b	a nor b
0	0	1
0	1	0
1	0	0
1	1	0

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)

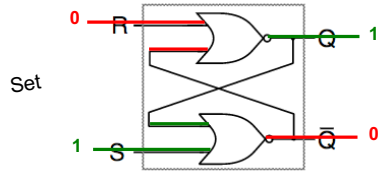


  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

Bascules

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)



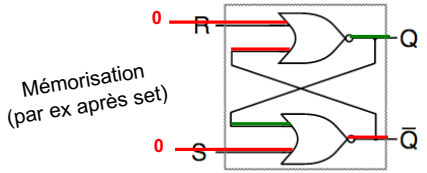
a	b	a nor b
0	0	1
0	1	0
1	0	0
1	1	0

  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

Bascules

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)



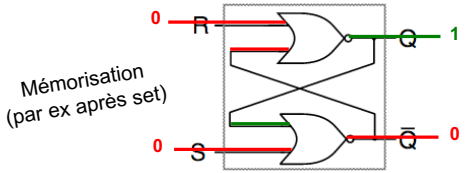
a	b	a nor b
0	0	1
0	1	0
1	0	0
1	1	0

  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

Bascules

- Bascule RS (suite) :
  - Réalisation possible (à base de portes NOR)



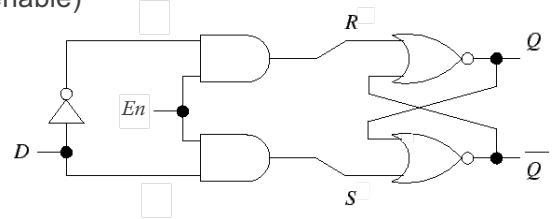
a	b	a nor b
0	0	1
0	1	0
1	0	0
1	1	0

  - Table de vérité

S	R	Q	$\bar{Q}$	
0	0	Q	$\bar{Q}$	← mémorise
1	0	1	0	← set
0	1	0	1	← reset
1	1	0	0	← interdit

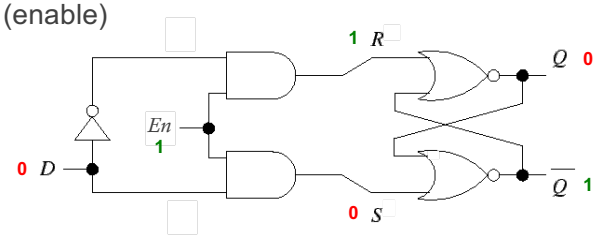
Bascules

- Et si on souhaite pouvoir charger et mémoriser une donnée D ? → **Bascule D**, avec entrée de commande En (enable)



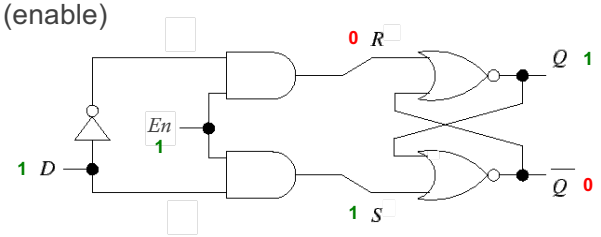
Bascules

- Et si on souhaite pouvoir charger et mémoriser une donnée D ? → Bascule D, avec entrée de commande En (enable)



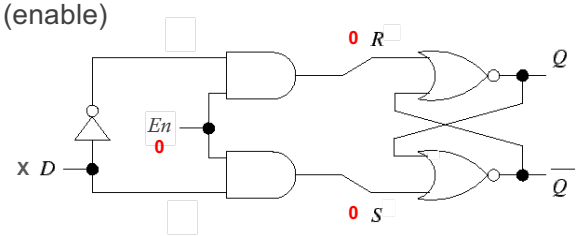
Bascules

- Et si on souhaite pouvoir charger et mémoriser une donnée D ? → Bascule D, avec entrée de commande En (enable)



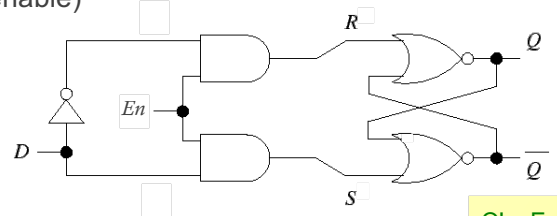
Bascules

- Et si on souhaite pouvoir charger et mémoriser une donnée D ? → Bascule D, avec entrée de commande En (enable)



Bascules

- Et si on souhaite pouvoir charger et mémoriser une donnée D ? → Bascule D, avec entrée de commande En (enable)

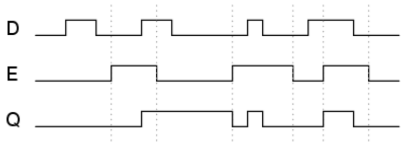


Q' = En . D + $\overline{\text{En}}$. Q

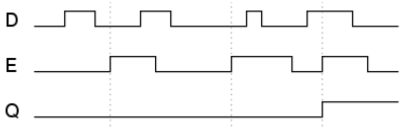
- Si En vaut 1, Q prend la nouvelle valeur de D (latch "transparent")
- Si En vaut 0, Q garde son ancienne valeur

Synchronisation par horloge

- Mais un tel dispositif est donc sensible au ***niveau*** de son entrée En



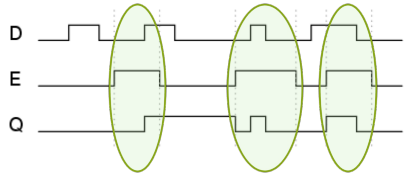
- Or il peut être nécessaire d'avoir des éléments mémorisants sensibles ***aux fronts*** (montants ou descendants) d'une entrée de contrôle



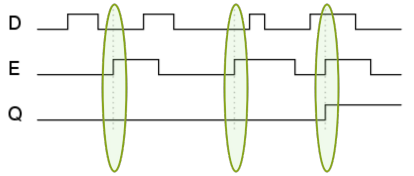
http://www.allaboutcircuits.com/textbook/digital/chpt-10/edge-triggered-latches-flip-flops/

Synchronisation par horloge

- Mais un tel dispositif est donc sensible au ***niveau*** de son entrée En



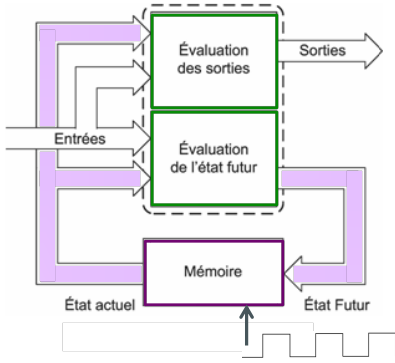
- Or il peut être nécessaire d'avoir des éléments mémorisants sensibles ***aux fronts*** (montants ou descendants) d'une entrée de contrôle

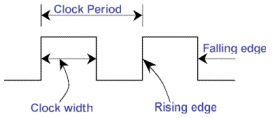


http://www.allaboutcircuits.com/textbook/digital/chpt-10/edge-triggered-latches-flip-flops/

Synchronisation par horloge

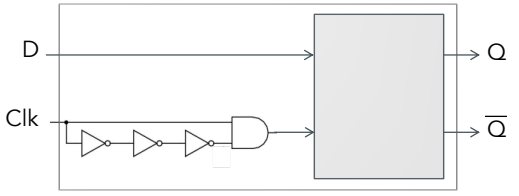
- En effet, rappelons que nous nous intéressons ici à la conception de circuits séquentiels synchrones





D flip-flop

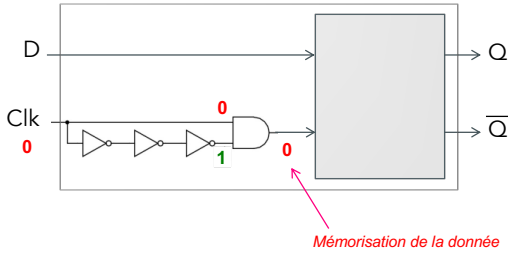
- On utilise alors des composants de type bascule D sensible aux fronts (usuellement montants) d'horloge, D flip-flop :
  - Un détecteur de fronts (montants ou descendants) de l'horloge pourra être placé sur l'entrée En, par ex.



Exploite le temps de propagation dans les portes NOT

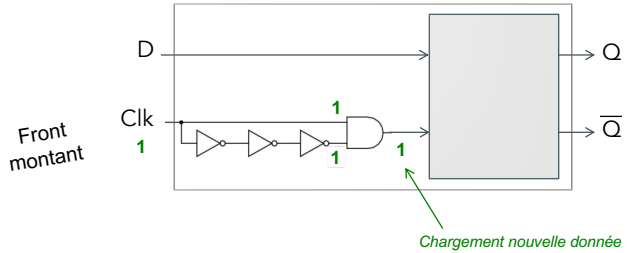
D flip-flop

- On utilise alors des composants de type bascule D sensible aux fronts (usuellement montants) d'horloge, D flip-flop :
  - Un détecteur de fronts (montants ou descendants) de l'horloge pourra être placé sur l'entrée En, par ex.



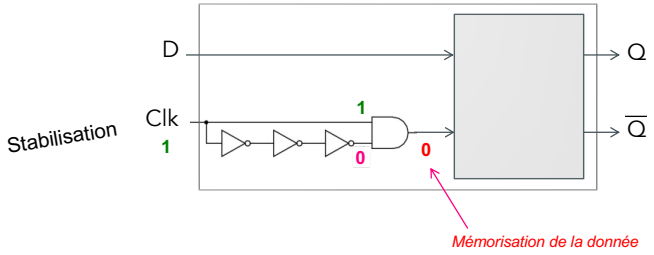
D flip-flop

- On utilise alors des composants de type bascule D sensible aux fronts (usuellement montants) d'horloge, D flip-flop :
  - Un détecteur de fronts (montants ou descendants) de l'horloge pourra être placé sur l'entrée En, par ex.



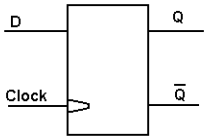
D flip-flop

- On utilise alors des composants de type bascule D sensible aux fronts (usuellement montants) d'horloge, D flip-flop :
  - Un détecteur de fronts (montants ou descendants) de l'horloge pourra être placé sur l'entrée En, par ex.

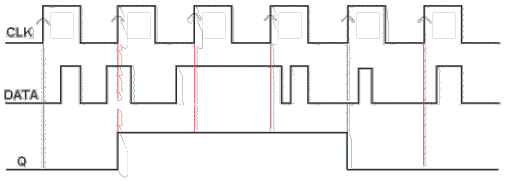


D flip-flop

- Le symbole du D flip-flop, sur fronts montants, est le suivant :



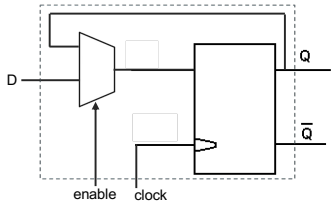
- Son comportement est par exemple :



http://www.hobbyprojects.com/flip_flop/edge_triggered_flip-flops.html

D flip-flop

- Enfin, il existe diverses variantes :
  - Avec signal de chargement (enable, ou load)



  - Et même avec signal de reset (remise à 0)

